
Smart Grid Energy Monitoring and Billing

A Lambda Architecture Pipeline

Use Case 3 · Kafka, Spark Structured Streaming, Airflow,
Parquet on S3-compatible storage, PostgreSQL

Group member	Registration number
Ravindu full name	reg. no.
Amami full name	reg. no.
Oshani full name	reg. no.

Repository	https://github.com/Raviyaa27/smartgrid-lambda-pipeline
Simulated clock	1 simulated day = 300 real seconds (288×), from 2026-01-01
Date	submission date

All figures in this report were measured on the running system unless stated otherwise. Domain quantities (dates, windows, lag) are in simulated time; machine quantities (run times, latency) are in real time, and each figure says which.

1 Use case and requirements

We chose **Use Case 3, Smart Grid Energy Monitoring & Billing**. A utility wants real-time visibility of grid load and solar contribution from smart meters, reconciled daily against tariff and billing data that arrives once a day. The business question is:

What is the current grid load and renewable contribution by zone, and what will each household's bill look like once daily tariff data is applied to their consumption?

This is two questions with incompatible service requirements, and separating them is the basis of every decision that follows (Table 1). Q1 is operational: a control room wants a number that is fresh and approximately right. Q2 is financial: a bill is regulated, disputable and must be reproducible years later.

Table 1: The business question decomposed. The two halves differ on every axis that matters for architecture.

	Q1 – Grid operations	Q2 – Billing
Question	Current load and renewable mix by zone	Household charge once the day's tariff is applied
Freshness	≤ 60 s	T+1 (next simulated morning)
Tolerance for error	High: a late reading moves a zone total by a fraction of a percent	Zero: exact to LKR 0.01 and reproducible on demand
Retention	Hours to days	Years (dispute and regulatory window)
Recomputation	Never needed	Required: backdated tariffs, late meter uploads, subsidy changes
Cost of being wrong	A slightly stale dashboard	A wrong charge, a refund, a compliance breach

From this we derived the requirements the system is built and tested against:

- R1** Zone load and renewable share visible within 60 s of a reading, labelled *provisional*.
- R2** Bills exact to the cent, reproducible, and never published provisionally.
- R3** Any past day can be *restated* (recomputed and reissued) with its original kept for audit.
- R4** Late and retransmitted readings are counted once, on the day they were taken.
- R5** Reference data that fails quality checks never produces a bill (fail closed).
- R6** Every stage is observable: logged, measured, and alerting on missing data, errors and low renewable share.

1.1 Simulated clock

To show several daily cycles, including restatement, in one session, time is compressed **288×: one simulated day lasts 300 real seconds**, starting at 2026-01-01. Seven simulated days take 35 real minutes. Each meter reports every 2 real seconds, which is one reading per **9.6 simulated minutes**. That gives about 30,000 readings per simulated day for our 200 households. 288× is the fastest ratio at which settling day N reliably finishes before day $N+1$ ends (Section 7.4).

Two rules make the clock trustworthy. First, *no component reads the wall clock to decide what day it is*. Event times, partition keys, window boundaries and settlement dates all derive from one **SimulatedClock**. Second, the clock's anchor is stored once, in `ops.sim_clock`, and read by every process. An early version anchored each process at its own start time. Two sources started 30 real seconds apart then disagreed by 2.4 simulated hours about the date, which would have broken the join between the two sources at every midnight.

2 Architecture decision: Lambda, not Kappa

Lambda [1] runs a batch layer that recomputes from an immutable master dataset alongside a speed layer that covers the batch layer’s latency. Kappa [2] keeps only the stream, and is the right default for most streaming systems: one engine, one code path, and reprocessing by replaying the log. The burden of proof is on choosing Lambda. We compared the two on the four axes the brief names, sized at production scale for a utility of this kind. That scale is 10^6 meters $\times$ 1 reading/minute = 1.44×10^9 events/day, about 288 GB/day raw, and about 736 TB over a 7-year dispute and retention window.

Table 2: Lambda versus Kappa on the axes that decide this use case.

Axis	Kappa (single streaming path)	Lambda (speed + batch)
Latency	Meets Q1 and Q2 from one stream. <i>Kappa’s strongest axis.</i>	Speed layer meets Q1; batch layer meets Q2.
Replay	Restating one day needs the log retained for years, or an external archive read by a separate job, which is Lambda under another name.	Re-run one date-parameterised job over one immutable Parquet partition. Targeted and bounded.
Cost	≈ 736 TB on broker disk, $\times 3$ replication $\approx$ 2.2 PB of hot storage , for a handful of restatements per year.	Parquet compresses this data 8–10 $\times$: ≈ 75 TB of cold object storage . Kafka keeps only a 7-day recovery window.
Consistency	Achievable, but the serving store is mutated continuously; proving a March bill at a November audit means reasoning about past stream state.	The batch layer is the declared system of record: deterministic output from immutable input. Reproducing a bill is re-running one job.

Decision: Lambda. The argument is not that Lambda is more capable. It is that this use case contains *two different consistency contracts*, and Lambda lets each be met on its own terms. The speed layer serves Q1 from the live stream. The batch layer serves Q2 by recomputing each day from an immutable master dataset. A serving layer merges them under an explicit rule:

Figure requested	Served from
Zone metrics for a <i>past</i> day	batch.* (settled): the batch view always wins
Zone metrics for <i>today</i>	speed.* (provisional), labelled PROVISIONAL
A household bill	batch.* only; no provisional bill is ever published

2.1 Rejected alternatives

- **Kappa.** Rejected on replay, cost and audit together. Serving a years-long restatement window needs either ≈ 2.2 PB of replicated broker storage for a rarely used capability, or a long-term archive with its own batch reader. The second *is* Lambda with the labels removed. *Kappa would win* if billing left the scope, or if disputes were limited to a few days, so that Kafka retention could cover the whole window.
- **Kappa with the tariff as a compacted topic.** This is the strongest Kappa variant: the daily tariff becomes a stream–table join. It solves the join, which was never the hard part. Restating March after a backdated tariff still means replaying, and so retaining, March’s readings.
- **Batch only.** Satisfies Q2 fully but fails Q1: a daily job cannot meet a 60-second freshness target or raise a low-renewable alert in time.

2.2 Answering Lambda’s known weakness

Kreps’ critique of Lambda [2] is that the same logic written for two engines inevitably diverges, and it is correct as stated. We answered it structurally, not by discipline: (i) both layers run on **one**

engine, Spark 3.5.9, with the same connector jars; (ii) validation rules are declared **once**, as **data** (a `FieldSpec` table), and a row-by-row validator and a vectorised validator are both thin adapters over it, imported by both layers; (iii) a **fuzz test** over 4,800 damaged messages holds the two validators to identical verdicts. It found a real defect: an infinite timestamp crashed both validators, which in Spark would have become a poison message replayed forever; and (iv) the approximation the speed layer makes is **measured**, not asserted, by reconciling it against every settlement (Section 7).

3 System architecture

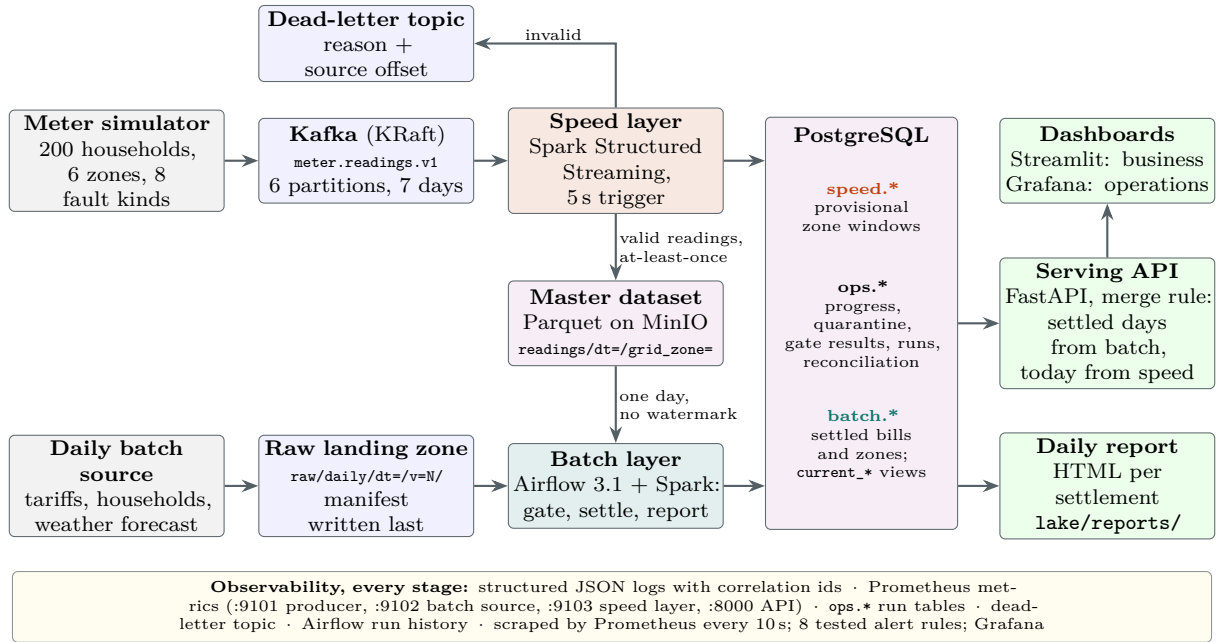


Figure 1: Architecture. The top row is the speed path (Q1), the bottom row the batch path (Q2). They share the Parquet master dataset, which the speed layer writes and the batch layer recomputes from. PostgreSQL keeps provisional (**speed**) and settled (**batch**) figures in separate schemas, so a consumer cannot confuse them. Every component reads one simulated clock, anchored in `ops.sim_clock`.

Speed path. The meter simulator publishes JSON readings to Kafka, keyed by `meter_id` so that each meter’s readings stay in order within one partition. A Spark Structured Streaming job validates every message with the shared validator. It sends rejects to a dead-letter topic with their reason and source offset. It appends valid readings to the master dataset, and maintains per-zone, 15-minute figures in `speed.zone_metrics`.

Batch path. Once per simulated day the batch source publishes an immutable, versioned *drop*: the tariff schedule, each household’s tier and subsidy flag, and a weather forecast per zone. It is complete only when its manifest exists. An Airflow DAG waits for that drop and for the archive to pass the end of the day. It then gates the drop and runs a Spark settlement over the whole day. Each settlement writes a new *run*, which is never overwritten.

Storage. Kafka is transport and short-term recovery (7 days), *not* the system of record. That is Parquet on MinIO, partitioned by `dt` and `grid_zone`. MinIO speaks the S3 API, so moving to AWS changes an endpoint, not code. PostgreSQL holds only aggregates and bills, in three schemas that encode the merge rule in the storage layer itself.

4 Technology stack and justification

Each choice is justified against this use case’s constraints rather than general popularity (Table 3), using the tools’ documented guarantees [6, 4, 5]. One theme runs through them: *the batch layer’s ability to recompute a day exactly* is the requirement most choices serve.

Table 3: Technology stack. Every component runs from one `docker compose up`; versions are pinned.

Layer	Choice	Why, for this use case	Rejected
Ingestion	Kafka 7.6 (KRaft), 6 partitions keyed by meter, 7-day retention	A replayable log. The speed layer restarts from its checkpointed offsets. Per-meter ordering where deduplication needs it. Retention sized to <i>recovery</i> , not restatement: the configuration line that encodes the Lambda choice.	RabbitMQ (no replay); Kinesis (no local reproducibility)
Stream processing	Spark 3.5 Structured Streaming	Native event-time windows and watermarks for late meter reads; exactly-once effects via checkpoints and idempotent sinks; the same engine as the batch layer , which contains Lambda’s drift risk.	Storm (no event time, no batch reuse); Flink (outside the named stack)
Batch processing	Spark 3.5 (batch), same image and jars	One artefact, two modes. Settlement re-validates with the same code the speed layer used.	A separate pandas job
Orchestration	Airflow 3.1	Sensors for the late daily file; per-task retries separating transient from fatal errors; the quality gate as a task that fails closed; run history as the settlement audit trail; restatement is re-triggering the DAG for a date.	cron (no dependencies, no history); Dagster (outside the stack)
Master dataset	Parquet on MinIO (S3 API)	Columnar and compressed (8–10×); partitioning by date makes “recompute one day” read exactly one partition; immutable input makes bills reproducible.	HDFS (a NameNode for no benefit); raw rows in PostgreSQL
Serving store	PostgreSQL 16; schemas speed , batch , ops	Serving queries are relational and ad hoc (household $\bowtie$ tariff $\bowtie$ zone). Serving volume is aggregates, orders of magnitude below raw readings.	Cassandra (no ad hoc joins; query-first modelling)
Serving & dashboards	FastAPI; Streamlit (business); Grafana (operations)	One API applies the merge rule so no consumer can bypass it; separate business and operations views for separate audiences.	–
Observability	JSON logs, Prometheus, Grafana	Metrics per stage with a pull model; logs that machines can filter by run, batch or correlation id.	Unstructured logs

Versions were chosen, not defaulted. Airflow 2 reached end of life in April 2026, so we used Airflow 3.1. Its dependency constraints pin PySpark 4.0.1, while the speed layer runs 3.5.9. We therefore install PySpark 3.5.9 separately in the Airflow image, so that both layers run exactly the same engine. Otherwise the “one engine” argument above would have been silently false.

5 Implementation

5.1 Simulated data sources

Streaming source. Each of 200 households has a meter in one of 6 grid zones and a tariff tier. Consumption follows a daily load curve normalised so a household’s average draw is its true base load. Rooftop solar is zero at night, peaks at noon, never exceeds rated capacity, and follows each zone’s *actual* weather. The daily drop publishes a *forecast*, so forecast and outcome differ as they do in reality. Energy is integrated over the simulated interval a reading covers (9.6 minutes). Integrating over real seconds would undercount by $288\times$, and a regression test pins this. The producer is idempotent with `acks=all`, and stamps a correlation id in each payload and Kafka header.

Daily batch source. At 00:30 simulated time it publishes the day’s drop as a new immutable version under `raw/daily/dt=D/v=N/`. A drop holds the tariff schedule, every household’s tier and subsidy flag, the weather forecast per zone, and a manifest. The tariff is *data*, not code. `TariffSchedule` carries block rates per tier, fixed charges, the solar export credit and the subsidy fraction, as decimal strings so no amount passes through a float. A backdated revision (`revise`) or a corrected republish (`publish`) becomes version $N+1$, beside the original. The manifest is written last, with each file’s SHA-256 and record count, so a half-uploaded drop is never read.

Faults are injected on purpose, with ground truth. A pipeline is only shown to be robust by feeding it the faults it claims to handle (Table 4). The stream carries the injected fault in an `injected_fault` Kafka header that the pipeline never reads. Detection can therefore be *measured* against ground truth.

Table 4: Injected faults and the component that handles each.

Fault	Injected as	Handled by
<i>Streaming source (realistic profile: 2.7% of messages)</i>		
Content (5 kinds)	missing field, negative value, 60 kWh spike, malformed JSON, unknown household	Shared validator → dead-letter topic with reason
Duplicate	The same reading retransmitted	Deduplicated by <code>event_id</code> : speed layer within watermark; batch layer over the whole day
Late	Held back, released after the watermark	Missed by the speed layer (by design), recovered by settlement
Backfill	A meter offline for hours, then uploading its backlog	As late: counted on the day it was taken
<i>Daily batch source (drawn per day from a seed)</i>		
Late / missing drop	Arrives after its day ends / never arrives	Airflow sensor waits, then times out and fails the run
Source corruption (5)	negative rate, unknown tier, missing or duplicated households, invalid record	Quality gate, validity checks: fails closed
Transit corruption	File truncated in transit	Quality gate, integrity: checksum and record count

5.2 Speed layer

Two streaming queries read the same topic, each with its own checkpoint:

- **ingest** (stateless, `foreachBatch`) validates each micro-batch with the shared vectorised validator, as a pandas function inside Spark. It appends valid readings to the master dataset, partitioned by `dt` and `grid_zone`, and sends rejects to the dead-letter topic. It records each micro-batch in `ops.*` keyed by (simulation, batch id), so a replayed batch after a crash is recorded once.
- **zone_metrics** (stateful, event time [3]) applies a 30-simulated-minute watermark and deduplicates by `event_id` within it. It then aggregates 15-minute tumbling windows per zone: consumption, solar generation, net energy, average grid load, renewable share, readings and meters reporting. Changed windows are *upserted* on (zone, window start), so a replay converges rather than double counts.

A design correction found while building. The obvious design deduplicates readings before archiving them. In a stream, deduplication must be bounded by a watermark. A watermark-bounded operator also *discards every reading older than the watermark*, and those late readings are exactly what the batch layer exists to recover. The archive therefore keeps every valid reading, written at-least-once, and deduplication happens where it is safe: in the real-time view, and over the whole day in settlement (recorded as an ADR amendment).

5.3 Batch layer

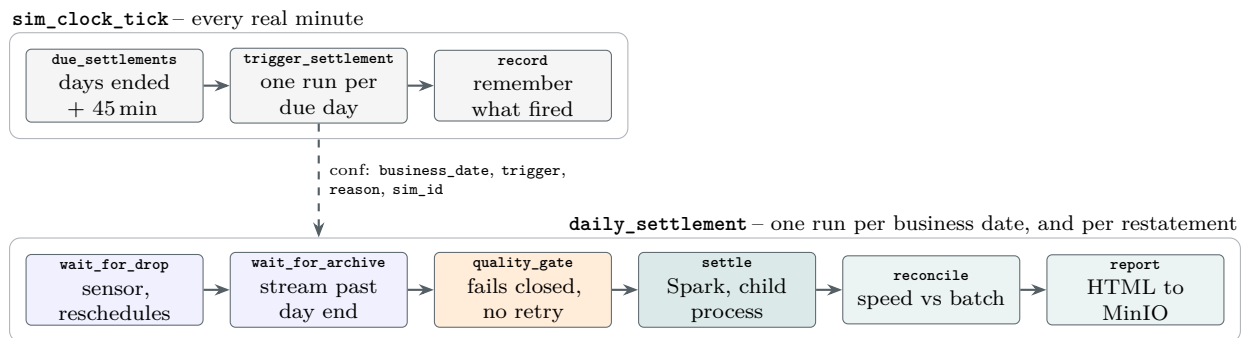


Figure 2: The two Airflow DAGs. A simulated day (300s) is shorter than Airflow’s practical scheduling granularity, so a clock-tick DAG triggers settlement with the simulated date as a parameter.

Triggering. Airflow’s logical dates are wall-clock time, and a simulated day lasts 300 real seconds, so settlement cannot use Airflow’s own schedule. The `sim_clock_tick` DAG runs every minute. It triggers `daily_settlement` once for each simulated day that has ended plus 45 simulated minutes of grace, enough for the watermark to close the day’s last windows. Each trigger uses a deterministic run id, `settle__<sim>__<date>`, so triggering a day twice is a no-op.

Settlement (`smartgrid.batch.settlement`) is a Spark job started as a child process of the Airflow task. The JVM therefore starts clean and exits with the job. It:

1. re-checks the drop against the quality gate itself, so no caller can bill from a bad drop;
2. reads the day’s Parquet partition and **re-validates it under today’s rules**, so fixing a rule and restating a day applies the fix to history;
3. deduplicates by `event_id` over the *whole day, with no watermark*, keeping the late readings the speed layer missed;
4. recomputes the 15-minute zone windows with *the same function* the speed layer uses, so reconciliation compares like with like;
5. **joins the two sources:** household totals with each household’s tier and subsidy flag, and the drop’s tariff schedule, to bill every household in exact decimal arithmetic; and zone totals with the zone’s weather forecast;

6. writes a snapshot of exactly the readings it settled to `lake/settled/dt=D/run=N/`, so any bill traces to its inputs;
7. commits bills and zone figures as a new run in **one transaction**.

A bill is the day’s net import priced on the tier’s block tariff (monthly blocks pro-rated to one day), plus the pro-rated fixed charge, minus the solar export credit, minus a 25% subsidy on the energy charge where the household is flagged.

Restatement without loss. Runs are append-only. Views `batch.current_*` expose the latest *successful* run for each day, and the serving layer reads those. A restated day therefore keeps its original bills beside the new ones. The auditor’s question “what did we bill, and why did it change?” is answered by the database, and the restated report states the change and its reason. Restating is one command: `scripts/settle.py --date D --restate --reason R`.

A defect found by running it. After a Docker restart, Airflow resumed a run that had been interrupted *before* a simulation reset. It then settled the new simulation’s half-finished day under the old run’s name. Every run now carries the id of the simulation it was triggered for, and the gate and the Spark job refuse a mismatch without retrying.

5.4 Serving layer

Consumers never query the stores directly. A FastAPI service applies the merge rule of Section 2 in one module (`serving/merge.py`), so no consumer can bypass it, and every figure it returns carries its **status** (SETTLED or PROVISIONAL) and **layer**. Building it exposed a case the rule had left out. A day that has ended is not settled at once: the 45-minute grace comes first, then the settlement itself. Serving nothing in that gap would blank yesterday every morning. Such a day is therefore served from the speed layer, labelled *awaiting settlement*.

Table 5: Serving API endpoints: GET requests under `/api/v1`. Interactive documentation at `/docs`.

Endpoint	Answers
<code>/zones/live</code>	Current load and renewable mix per zone, with each figure’s age
<code>/zones/daily?date=D</code>	A day’s totals per zone; the speed-vs-batch gap once settled
<code>/zones/{zone}/windows?from=D1&to=D2</code>	15-minute windows across days, each labelled by source
<code>/households/{id}/bills?from=D1&to=D2</code>	Settled bills, and why any other day has none
<code>/households/{id}/bills/{D}/history</code>	The original bill and every restatement of it
<code>/bills?date=D, /settlements, /reports/...</code>	A day’s bills by tier; run history; the daily report

A response from the live run, at simulated 2026-01-02 17:15, shows the rule at work:

```
GET /api/v1/zones/ZONE-C/windows?from=2026-01-01&to=2026-01-02
"days": [
  {"date": "2026-01-01", "status": "SETTLED", "layer": "batch", "settlement_run_id": 38},
  {"date": "2026-01-02", "status": "PROVISIONAL", "layer": "speed",
   "reason": "today: a day is settled only after it ends"} ]
```

The response then lists 91 settled windows for day 1 and 67 provisional ones for today.

- **Live means the latest *complete* window.** The newest window is still filling, and its load undercounts until every meter has reported. The API serves each zone’s latest window that the stream has passed, states its age in both clocks, and flags it **stale** beyond the 60s target. In the live run the view was 45.8 simulated minutes old (9.5s real): not stale.
- **No provisional bill, and no silent gaps.** A day without a settled bill is listed with its reason (“day in progress”, “awaiting settlement”), and `/bills` returns 404 until settlement. Bill history

shows restatements. For example, one household's day-1 bill was 792.77 LKR under drop v1 and 871.06 LKR after the restatement, with the regulator's reason attached.

- **Money is exact:** amounts are decimal strings ("871.06"), never floats.
- **Observed switching live.** Day 1 changed from PROVISIONAL to SETTLED (run 38), and its bills from 404 to 200, as soon as settlement committed. Nothing was restarted and no cache was cleared, because the API reads the `batch.current_*` views on every request.

It is tested at two levels. 32 unit tests cover the merge rule and the HTTP contract against an in-memory store. 6 integration tests run every SQL query against the real schema.

5.5 Code quality and testing

The code is one Python package (`smartgrid`) with modules for `common` (clock, schemas, shared validation, billing, storage), `producers`, `streaming`, `batch`, `serving` and `dashboard`. The DAG files only wire together functions that are tested without Airflow. The suite has **382 unit and integration tests**, and **8 Spark tests** that run inside the Spark image. They include the validator parity fuzz test, energy-model regressions, integration tests against MinIO under a throw-away prefix, billing arithmetic checked to the cent, and the alert-rule tests. Nine architecture decision records (ADRs) document each major choice with its rejected alternatives. Eight of their amendments record what we got wrong at first and how running the system showed it.

Reproducible with one command. Every component, including the simulated sources, runs from `docker compose up`; the host needs only Docker and Python. `make demo` (or `dev.ps1 demo`) builds the five images and starts a fresh simulation. It then walks through the outage, settlement, restatement and refused-drop scenarios, checks each claim of Section 7 against the running system, and ends with a PASS/FAIL table.

6 Observability design

The goal is that a failure is *detected* by a signal rather than a person, and *diagnosed* without attaching a debugger. Table 6 lists what is measured, how, and why.

Alert rules. Prometheus scrapes every component each 10s (pull, so a dead component shows as down rather than quiet) and evaluates eight rules (Table 7). Every threshold comes from a requirement or a measurement, not a guess, and is in real seconds, the clock operators work on. The rules are code, and they are **unit-tested with promtool**: synthetic series assert when each rule fires, and also when it must *not*. There is no solar alert at night, and no six-zone alert storm when the whole speed layer is stale, because `ZoneSilent` applies only while the stream itself is fresh. Breaking one input on purpose made the suite fail, so the tests do test.

The solar floor is exported by the API from the same setting the business dashboard uses, so the dashboard warning and the alert cannot disagree. Grafana shows it all on one provisioned dashboard (Figure 3). Its JSON is generated from a short script, so a panel change is a reviewable diff. Alertmanager was left out deliberately: nobody is paged in a demo, and adding it later changes routing, not the rules (ADR-0009).

Verified live. In a fresh simulation, every alert state change was logged as it happened:

- `ZoneSilent` fired for ZONE-C *alone*, 106s after the simulator took the zone offline. It resolved 21s after the zone returned and uploaded its backlog.
- `DropRefused` fired when the gate refused day 2's deliberately truncated drop. It resolved as soon as the republished drop passed, and the day then settled.
- `LowRenewableShare` went pending twice for ZONE-B at midday. The share never stayed below 30% for the required 20s, so it correctly did not fire: the `for` clause absorbs flicker that the dashboard's instantaneous warning shows.

Table 6: What is measured, how, and why.

Signal	How	Why
Structured logs	JSON lines from every service: timestamp, service, level and context. A <code>PipelineStage</code> wrapper logs each stage's records in, out and quarantined, and its duration.	Filterable by run, batch or date; every stage reports the same fields.
Tracing	A correlation id stamped by the producer in payload and Kafka header, carried into dead-letter records.	Follow one reading from meter to DLQ or archive.
Producer metrics (:9101)	Messages by outcome, delivery errors, late buffer, meters offline, simulated time, round duration.	Source health; the no-data alert input.
Batch source (:9102)	Drops published, last business date, publish errors.	A missing drop is visible before settlement waits on it.
Speed layer (:9103)	Records by outcome, late rows dropped, input and processing rate, micro-batch duration, last batch time, watermark, archived event time.	Freshness (R1), error rate, and whether the stream keeps up.
Run bookkeeping	<code>ops.ingest_batches</code> , <code>quarantine_counts</code> , <code>stream_progress</code> , <code>quality_gate_results</code> , <code>settlement_runs</code> , <code>reconciliation</code> .	A durable, queryable history; the batch layer's audit trail.
Dead-letter topic	Every rejected message with reason and source partition and offset; 14-day retention (longer than the source's 7 days).	Evidence survives until someone investigates.
Orchestration	Airflow run history; the settle task streams Spark's output and per-step timings into its log.	A slow or failed settlement is diagnosed from its log.
Serving API (:8000)	Requests and latency per route <i>template</i> (bounded label cardinality); a JSON log line per request with a request id echoed in <code>X-Request-ID</code> ; <code>/health</code> reports speed-layer lag against the 60 s target.	What consumers saw, and whether it was fresh.
Pipeline state (:8000)	Computed at each scrape from the serving store: each zone's data age, solar share, load and alert-hours flag; the speed layer's lag; days ended, settled, unsettled and refused. Ages in <i>real</i> seconds.	The business state no single component knows, on the same figures the dashboard shows.
Health checks	Docker health checks on Kafka, PostgreSQL and MinIO (bucket existence); dependents start only when healthy.	The stack starts in a known state.

Table 7: Alert rules (`infra/prometheus/rules`), with the basis of each threshold.

Rule	Fires when	Catches
<code>SpeedLayerStale</code>	real-time view > 60 s behind the clock (R1)	speed layer behind or stopped
<code>ZoneSilent</code>	one zone's data > 60 s old while the stream is fresh	feeder or network outage
<code>LowRenewableShare</code>	zone solar share < 30 % during 10:00–14:00	the use case's operational alert
<code>HighQuarantineRate</code>	> 5 % of readings rejected for 1 min (baseline 0.7 %)	upstream firmware or schema fault
<code>DropRefused</code>	the quality gate refused a day's drop	no bills until it is republished
<code>SettlementOverdue</code>	a day unsettled > 360 s after it was due (normal: ≈ 90 s)	missing drop, failed job
<code>ScrapeTargetDown</code> , <code>ServingStoreDown</code>	a component or PostgreSQL stops answering for 30 s	process or database down

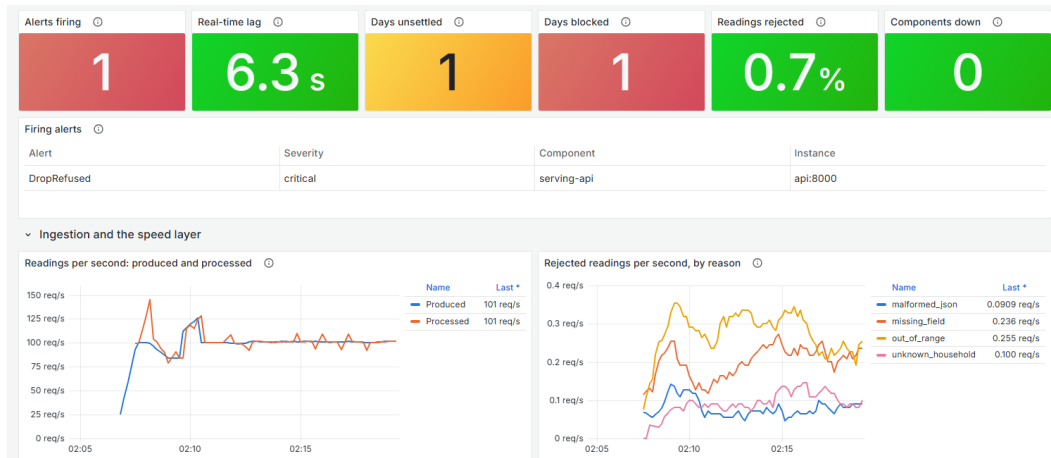


Figure 3: Grafana during the live run (top section). The tiles show status by colour *and* number: DropRefused fires for day 2’s refused drop, while the real-time view is 6.3s behind and 0.7% of readings are rejected (the baseline). Further panels cover lag, micro-batch times, solar share and load by zone, the settlement backlog and API traffic.

- **ScrapeTargetDown** fired for components still starting, and resolved 11–22s after each one’s first successful scrape.

Observability in use. These signals found four real faults during the build. (i) The watermark and stream-progress metrics showed the real-time view trailing the clock by 2.5 simulated hours at local[2]. At local[4] the lag fell to about 30 simulated minutes (≈ 6 s real). (ii) A corrupt drop was refused, and its findings (checksum and record-count mismatch) were recorded in `ops.quality_gate_results`. (iii) `ops.settlement_runs` exposed the cross-simulation run: its Airflow run id named the old simulation. (iv) A settlement that took over 7 minutes was diagnosed from per-step timings as memory starvation on the host, not a slow job.

7 Results

7.1 Correctness, end to end

Each layer was verified against the injected ground truth, on the running stack (Table 8).

Table 8: Verification against ground truth.

Check	Result
Stream fault detection	3,230 live messages: all 101 injected faults classified as designed; 0 false rejections .
Daily drop quality gate	11 business dates covering every fault kind, a revision and a recovery republish: verdict matched ground truth on 11 of 11 .
Speed layer, no loss	42,529 messages published = 42,529 processed , across a mid-stream restart that resumed from its checkpoint (micro-batch 9 after 8).
Speed layer, routing	311 rejected = exactly the injected content faults; 311 dead-lettered; 42,218 archived; no double writes.
Batch layer	Day 1 settled automatically: 28,245 archived readings, 269 retransmissions removed, 352 late readings recovered, 200 households billed, LKR 206,276.28.
Fail closed	Day 2’s drop was published truncated: the gate refused it on checksum and record count, and no bill was written . After the drop was republished, day 2 settled normally: 29,777 readings, 232 late, LKR 215,772.88.

7.2 The consolidated daily report

Every settlement publishes a self-contained HTML report (Figure 4). It answers the business question for the settled day: load and renewable contribution by zone, and what each tier and household is billed. It also states how reliable the real-time view was.

Daily settlement — 2026-01-01 SETTLED

Run 36 · restatement · tariff drop v2 · finished 2026-09-30 16:57:09 UTC (real time) · all figures in simulated time

Restatement. This run supersedes an earlier settlement of the same day. Total billed changed by LKR 2,721.52 (from 206,276.28). Reason: Regulator backdated revision: DOMESTIC_STD +10%

200

households billed

LKR 208,997.80

total billed

6,225.8 kWh

energy consumed

29.1%

renewable contribution

Grid by zone

Zone	Consumed kWh	Solar kWh	Renewable	Peak load kW	Forecast irradiance	Speed vs batch	Late readings
ZONE-A	991.4	242.2	24.4%	63.3	0.58	-1.12%	110
ZONE-B	1,232.3	194.3	15.8%	77.5	0.88	-1.25%	77
ZONE-C	1,707.2	358.6	21.0%	111.8	0.63	-0.68%	29
ZONE-D	790.3	397.9	50.4%	52.5	0.90	-0.94%	43
ZONE-E	720.9	365.7	50.7%	43.8	0.63	-1.32%	30
ZONE-F	783.8	251.0	32.0%	54.9	0.87	-0.70%	63

"Speed vs batch" is how far the real-time (provisional) view was from these settled figures. "Late readings" arrived after the real-time view's watermark: 352 in total, all included here.

Billing by tier

Tier	Households	Net import kWh	Billed LKR	Average LKR
DOMESTIC_HIGH	41	841.2	42,440.77	1,035.14
DOMESTIC_LOW	51	207.9	1,420.17	27.85
DOMESTIC_STD	86	766.4	24,605.74	286.11
INDUSTRIAL	22	3,098.5	140,531.12	6,387.78

42 households finished the day in credit (net exporters).

Figure 4: The daily report for 2026-01-01 after a restatement (run 36, drop v2). The highlighted note gives the change in total and its reason. "Speed vs batch" is the real-time view's measured error per zone. The report continues with the largest bills and data-quality counts.

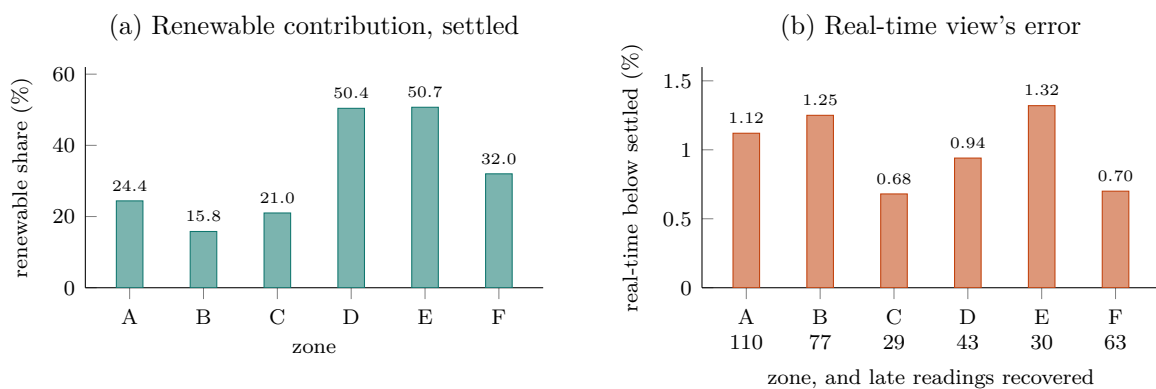


Figure 5: Day 1, 2026-01-01. (a) Solar share of consumption by zone. (b) How far the provisional real-time figure fell below the settled one: the speed layer under-reported every zone, by 0.7–1.3%. The cause is readings that arrived after its watermark, which the batch layer recovered (count under each zone).

The Lambda trade-off, measured. The speed layer was never more than 1.32% from the settled

figure in any zone. That is ample for grid operations (Q1). The reconciliation counts exactly which readings the real-time view never saw: 352 that arrived after its watermark, all recovered by settlement. Both layers apply the same validator and the same window function, so the gap is attributable to timing, not to diverging logic. That is precisely why billing (Q2) must come from the batch layer. An outage makes the point sharply. In the scripted demo, ZONE-C was taken offline for 90 real seconds, and on return its meters uploaded the missed readings, all behind the watermark. The real-time figure for ZONE-C’s day was therefore 36 % low. Settlement recovered 1,313 backfilled readings and corrected it, while the other zones stayed within 1.3 %.

7.3 Restatement

A regulator’s backdated +10% revision of the DOMESTIC_STD tariff for day 1 was published as drop v2, and the day was restated through Airflow (Table 9).

Table 9: Day 1 bills by tier: original settlement (run 35) against the restatement (run 36).

Tier	Households	Original (LKR)	Restated (LKR)	Change
DOMESTIC_HIGH	41	42,440.77	42,440.77	0.00
DOMESTIC_LOW	51	1,420.17	1,420.17	0.00
DOMESTIC_STD	86	21,884.22	24,605.74	+2,721.52
INDUSTRIAL	22	140,531.12	140,531.12	0.00
Total	200	206,276.28	208,997.80	+2,721.52

Only the revised tier changed, and the other three were identical to the cent. The tier’s energy charge rose by exactly 10.00% (27,211.09 to 29,932.61 LKR). Its *net* bill rose by 12.4%, because fixed charges (860.00) and solar export credits (6,186.87) are unchanged while being netted against a larger energy charge. Run 35’s bills remain in the database beside run 36’s.

7.4 Performance

Day 1 settled in **89 s real time from trigger to published report** (Table 10), comfortably inside the 300 s of a simulated day. Reading many small Parquet files dominates the cost; it is discussed in Section 8. The real-time view trails the simulated clock by about 30 simulated minutes (≈ 6 s real), within the 60-second freshness target.

Table 10: Settling one simulated day (28,245 readings in 309 Parquet files). Real seconds.

Airflow task	s	Spark job step	s
wait_for_drop, wait_for_archive	4	list the archive	9.1
quality_gate	2	read and re-validate	24.9
settle (incl. 24 s start-up)*	74	deduplicate	2.9
reconcile	2	aggregate	9.7
report	3	write snapshot	2.8
		bill and commit	0.1
Trigger to report	89	Spark total	49.5

*Starting the child process, the JVM and Python, and the job’s own re-check of the drop.

7.5 The business dashboard

A Streamlit dashboard answers the business question for a non-technical reader. It reads *only* the serving API, so it cannot disagree with the merge rule. Its five pages cover the current grid,

zone history across days, one settled day, household bills with their restatement history, and every settlement run. Two design rules make the provisional/settled distinction impossible to miss: (i) source is shown by colour *and* words, blue for SETTLED and orange for PROVISIONAL. That pair was validated as colour-blind safe (worst CVD ΔE 24.7 in light mode, 26.8 in dark, against a target of 8), and each source keeps its colour when the other is absent; (ii) load and solar share are always separate charts, never one chart with two y-axes. Every chart has a table view.

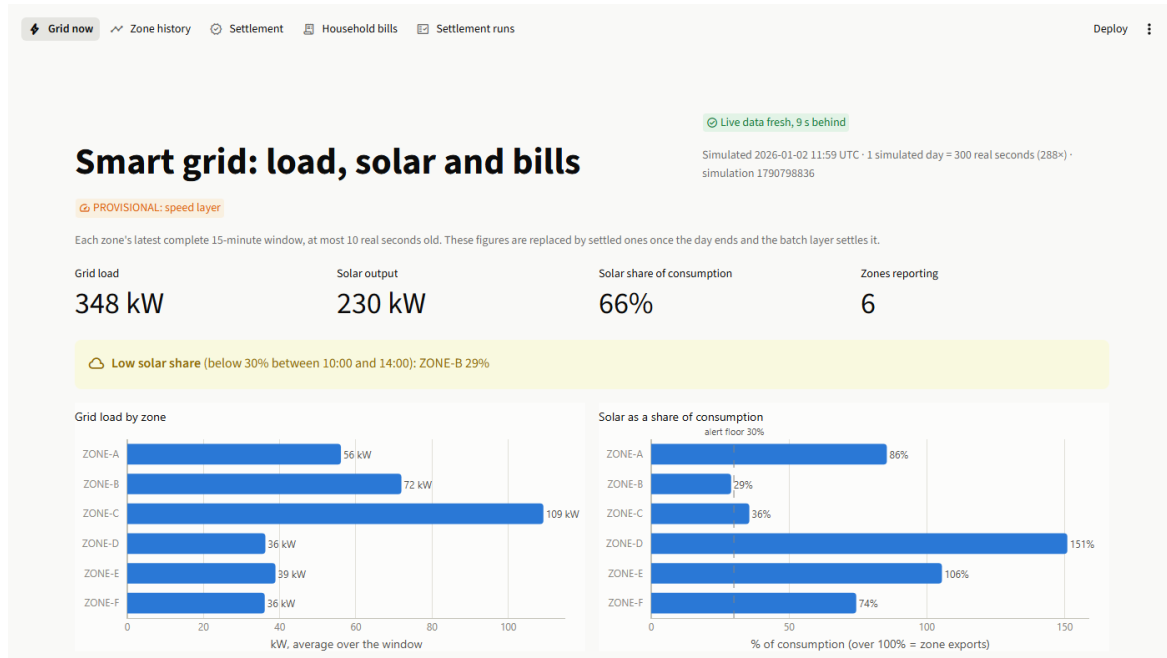


Figure 6: Grid now, at simulated 2026-01-02 11:59, during the live run. Each zone's latest complete window, 9s old in real time and labelled PROVISIONAL. ZONE-B triggered the low-solar warning at 29% against a 30% floor, which applies only between 10:00 and 14:00: at night every zone is at 0%, which is not news. The floor was calibrated on the simulation, where midday zone shares run from 25% to 180%.

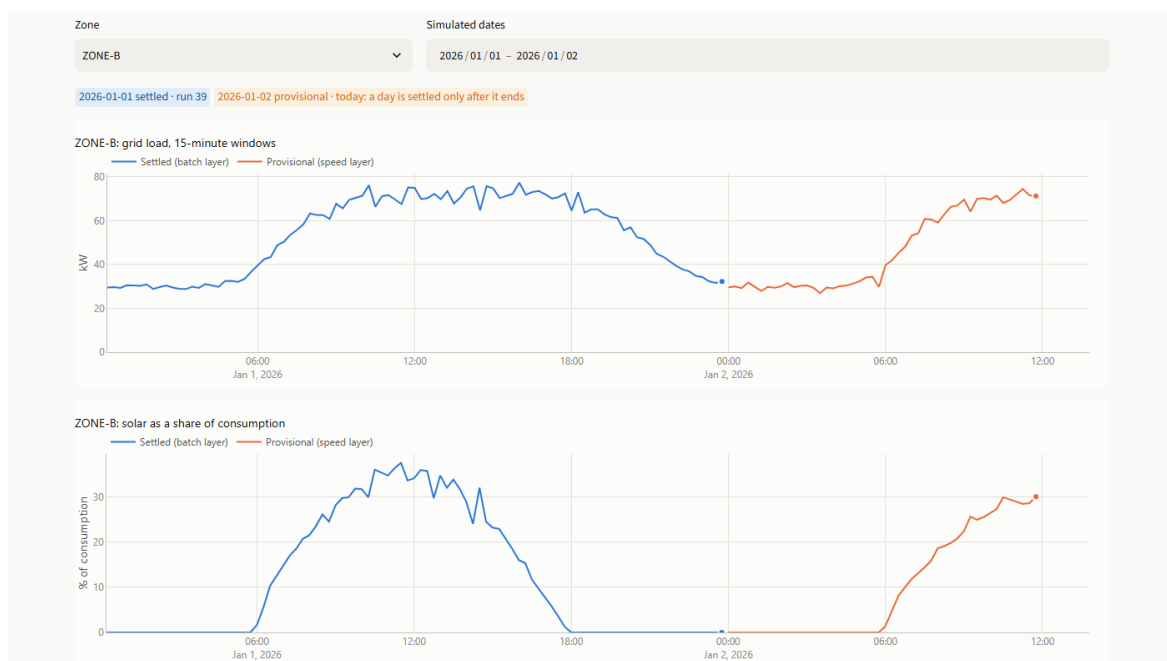


Figure 7: Zone history: the merge rule on one chart. Day 1 is settled (run 39, blue), recomputed by the batch layer with its late readings. Day 2 is today, provisional from the speed layer (orange). The day badges above the charts say which layer served each day.

The household page makes restatement visible to a billing clerk. In the same run, a backdated +10% DOMESTIC_STD revision was restated through Airflow. For household HH-00071 the page then listed both settlements of day 1: 825.67 LKR under drop v1 (run 39), and 907.24 LKR under drop v2 (run 40), marked as the bill now shown, with the regulator’s reason.

The operations counterpart, the Grafana dashboard, is shown in Figure 3.

8 Limitations, trade-offs and production scale

Deliberate simplifications of the demo.

- **Single Kafka broker, replication factor 1.** No fault tolerance. Production would use three brokers, replication 3 and `min.insync.replicas=2`.
- **Spark in local mode** (`local[4]`) in one container per layer, rather than a cluster.
- **Airflow standalone** with LocalExecutor, a UI without login, and its metadata in the serving PostgreSQL instance. Acceptable locally, never shared.
- **Tariff blocks are pro-rated to one day.** A real utility accumulates month-to-date consumption and charges the marginal block.
- **Compressed time exaggerates throughput:** 288 simulated days pass in one real day, so our throughput figures are not production figures.
- **Security:** development credentials in `.env`, no TLS. MinIO runs from a pinned archive image, because MinIO withdrew its public images, and it receives no security updates.

Trade-offs we accepted, with their measured cost.

- **Two processing paths** remain Lambda’s real cost: two deployments (a streaming container and the Airflow image) must pin the same Spark version. We hit exactly this when Airflow’s constraints pinned a different PySpark.
- **Python validation inside Spark**, rather than native expressions, buys one provably shared rule set at a throughput cost. Reading and re-validating a day is the largest step in settlement (24.9 s).
- **Small files.** The speed layer writes a file per zone per micro-batch: 309 files for one simulated day. Settlement reads them rather than rewriting the raw partition, because late readings can still arrive after a day is settled. Without a table format, rewriting that partition could delete a file written during the rewrite. Each run instead writes one compacted snapshot of what it settled.
- **The watermark** trades completeness for freshness. At 30 simulated minutes the real-time view is within 1.3% of settled. A longer watermark would narrow the gap and delay the view.
- **Resources.** The full stack needs about 6 GB of memory while settling. On a starved host, settlement slowed from 90 s to over 7 minutes, and the Docker runtime once failed. The README states this requirement.

What we would do differently at production scale.

1. **A table format (Apache Iceberg or Delta Lake) over the master dataset and landing zone.** Restatement and compaction become transactional; time travel replaces our version folders and manifests with the same semantics; and small files are compacted in place.
2. **Compile the FieldSpec rules to native Spark expressions**, keeping one declaration but removing the Python round trip.
3. **A schema registry** with Avro or Protobuf instead of JSON, so schema changes are checked at the producer rather than discovered in the dead-letter topic.
4. **Re-evaluate Flink with an Iceberg sink.** If a stream processor can recompute targeted historical partitions cheaply, the batch layer’s reason to exist weakens and Kappa becomes viable. This is the condition under which we would revisit this decision.

5. **Cluster deployment:** Spark on Kubernetes, Airflow with a Kubernetes executor and its own database, and managed object storage.

9 Individual contributions

The project was planned and reviewed jointly. Design decisions were agreed in group discussion and recorded as ADRs, and each member reviewed the others' code before it was merged. Primary responsibilities were divided as follows.

Table 11: Division of work.

Member	Primary responsibilities	Share
Ravindu	Overall architecture and the Lambda-versus-Kappa decision; lead author of the ADRs. Infrastructure: the Docker Compose stack (Kafka, PostgreSQL, MinIO) and the Spark image. The shared foundation: simulated clock, configuration, shared validation module and its parity tests. The speed layer (Spark Structured Streaming). End-to-end integration and verification runs; report editing.	36%
Amami	The streaming source: smart-meter simulator, energy and solar model, fault injection. The billing model: tariff schedule, block tariffs, exact decimal arithmetic, subsidy and export credit. Observability: Prometheus metrics, Grafana dashboards and alert rules. The Streamlit business dashboard. Report sections on data sources, observability and results.	32%
Oshani	The daily batch source: versioned drops, manifests and the quality gate. The batch layer: the Spark settlement job, Airflow DAGs, reconciliation, daily report and restatement. The serving API and its merge rule. Report sections on the batch and serving layers, and limitations.	32%

References

- [1] N. Marz and J. Warren, *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*. Manning, 2015.
- [2] J. Kreps, "Questioning the Lambda Architecture," O'Reilly Radar, July 2014.
- [3] M. Kleppmann, *Designing Data-Intensive Applications*. O'Reilly, 2017.
- [4] Apache Software Foundation, *Structured Streaming Programming Guide*, Apache Spark 3.5 documentation.
- [5] Apache Software Foundation, *Apache Airflow 3.1 documentation*.
- [6] Apache Software Foundation, *Apache Kafka documentation*, including KRaft mode.